Security Assessment Report

DAMM Capital

August 2025

Prepared for DAMM Capital

Table of content

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
DAMM Capital	https://github.com/DAMM-Cap/UniswapV4-Zodiac-Roles	d75c9d0 (original commit) 91de30a (fix commit)	Solidity

Project Overview

This document describes the manual code review findings of **DAMM Capital**. The following contract list is included in our scope:

`DAMM-Cap/UniswapV4-Zodiac-Roles/src*`

The work was undertaken from **July 28, 2025,** to **August 4, 2025**. During this time, Certora's security researchers performed a manual audit of all the Solidity contracts and discovered several bugs in the codebase, which are summarized in the subsequent section.

Protocol Overview

DAMM Capital's UniswapV4 Zodiac roles are custom verification contracts for Zodiac Roles V2 (v2.1) that enable precise permission control for Uniswap V4 interactions. They solve specific limitations in the current Zodiac Roles implementation that prevents proper handling of Uniswap V4 permissions. The implementation was not possible using the Zodiac JS SDK due to these limitations. These verifier contracts function as calldata struct decoders to enable fine-grained transaction verification.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	4	4	3
Informational	2	2	2
Total	6	6	5

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

certora

DAMM CAPITAL

Detailed Findings

ID	Title	Severity	Status
L-01	Missing data size checks in the majority of contracts	Low	Fixed
L-02	Size check during liquidity decrease is wrong	Low	Fixed
L-03	Swaps and mints lack slippage validation	Low	Acknowledged
L-04	System susceptible to address clashes	Low	Fixed
I-01	Forgotten Foundry import is still present	Informational	Fixed
I-02	Missing support for exact out swaps	Informational	Fixed

Low Severity Issues

L-01. Missing data size checks in the majority of contracts

Severity: **Low**	Impact: **Low**	Likelihood: **Medium**
Files: src/UniswapV4MintStructVerifier.sol src/UniswapV4SettleAllStructVerifier.sol src/UniswapV4SettlePairStructVerifier.sol src/UniswapV4SwapExactInSingleStructVerifier.sol src/UniswapV4SweepStructVerifier.sol src/UniswapV4TakeAllStructVerifier.sol src/UniswapV4TakePairStructVerifier.sol	Status: Fixed	

Description: The Zodiac roles contracts are all meant to verify the incoming data slice and it's decoded parameters for UniswapV4. The call comes with a data start location and a `size` variable for correct slicing.

However, currently only the `UniswapV4DecreaseLiquidityStructVerified.sol` contract verifies that the incoming size is large enough for decoding:

```
JavaScript
    function check(
         address to,
         uint256,
         bytes calldata data,
         uint8, // 0 = Call, 1 = DelegateCall
         uint256 location,
         uint256 size,
         bytes12 extraData
     ) external view returns (bool, bytes32) {
         /// check that size is at least 192 bytes
```

```
        if (size < 0xC0) return (false, Lib.INVALID_ENCODING);

        try this.decode(data, location, size) returns (
            uint256 tokenId, uint256 liquidity, uint128 amount0Min, uint128 amount1Min, bytes
memory hookData
        ) {
            /// check if tokenId is owned by avatar using ERC721
            if (IERC721(to).ownerOf(tokenId) != IModifier(msg.sender).avatar()) {
                return (false, Lib.INVALID_TOKEN_ID);
            }
        } catch {
            return (false, Lib.INVALID_ENCODING);
        }

        return (true, 0);
    }
```

Every other verification contract is missing this check, thus incoming incorrect calls could fail decoding.

Recommendations: Introduce proper data size checks in every verifier or remove the current one, as the decoding will fail anyway. The former option saves more gas and would add clarity via the error code.

Customer's response: Fixed in 727537d.

L-02. Size check during liquidity decrease verification is wrong

Severity: **Low**	Impact: **Low**	Likelihood: **Medium**
Files: src/UniswapV4DecreaseLiquidityStructVerifier.sol	Status: Fixed	

Description: The `UniswapV4DecreaseLiquidityStructVerifier.sol` contract does a necessary check on the passed `size` variable in order to verify that the data slice during decoding would be correct and would contain all necessary variables:

```JavaScript
    function check(
        address to,
        uint256,
        bytes calldata data,
        uint8, // 0 = Call, 1 = DelegateCall
        uint256 location,
        uint256 size,
        bytes12 extraData
    ) external view returns (bool, bytes32) {
        /// check that size is at least 192 bytes
        if (size < 0xC0) return (false, Lib.INVALID_ENCODING);

        try this.decode(data, location, size) returns (
            uint256 tokenId, uint256 liquidity, uint128 amount0Min, uint128 amount1Min, bytes
memory hookData
        ) {
            /// check if tokenId is owned by avatar using ERC721
            if (IERC721(to).ownerOf(tokenId) != IModifier(msg.sender).avatar()) {
                return (false, Lib.INVALID_TOKEN_ID);
            }
        } catch {
            return (false, Lib.INVALID_ENCODING);
        }

        return (true, 0);
```

```
    }
```

The check itself, however, is wrong since it checks that the size is greater than 192 bytes when it should be 224. That is because the encoded data contains a dynamic `bytes` field for the `hookData` that would go to UniswapV4. Even if that data is empty, it would create 2 fields in the head of the calldata for the offset and the length. With this in mind, the layout of the data looks like:

```
| 32 ARRAY_OFFSET | 32 tokenId | 32 liquidity | 32 amount0min |

32 amount1min | 32 hookDataOffset | 32 hookDataSize | arbitrary hookData |
```

Collecting all of the known size parameters we get 224 bytes minimum, considering the `hookData` is empty. Even if empty, it will have a size field and an offset field.

Recommendations: Change the check from comparing the size to 192 to comparing it to 224 bytes (0xE0 in hex)

Customer's response: Fixed in d461588.

L-03. Swaps and mints lack slippage validation

Severity: **Low**	Impact: **Low**	Likelihood: **Medium**
Files: src/UniswapV4MintStructVerifier.sol src/UniswapV4SwapExactInSingleStructVerifier.sol	Status: Acknowledged	

Description: Both the position mint and swapping verifiers include necessary checks for the fee tiers, pools, currencies, etc., aiming at avoiding high impact errors. Even though seemingly intentionally omitted, the lack of verification for valid slippage during the Safe transaction ensures that there would not be any inconvenient losses due to market conditions.

Since the Safe would call UniswapV4's routers and managers, which do not enforce it, such swaps/mints could occur and lead to loss of value.

Recommendations: Ensure there is slippage provided when such transactions are executed

Customer's response: This is an intended footgun for operator.

L-04. System susceptible to address clashes

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: https://github.com/DAMM-Cap/UniswapV4-Zodiac-Roles/blob/main/src/Lib.sol	Status: Fixed	

Description: The extraData passed in to the check function contains the token addresses. A 12 byte storage is used to store the truncated token addresses, with 6 bytes dedicated to each token. The issue with this is if multiple tokens are present with the same leading 6 bytes, the contract can no longer distinguish between them. This means tokens not meant to be whitelisted can still be used to provide liquidity.

Vanity addresses are often mined with leading zeroes for gas efficiency, or just for showing off and protocols often use these mined addresses to deploy tokens. A 6 byte vanity address would involve 12 leading zeroes. The winner of the uniswap address mining challenge here was already able to mine vanity addresses with 15 desired characters, so 12 leading desired characters are already possible. So if multiple protocols dedicate resources to mine addresses with 12 leading zeroes, the contract system can have issues distinguishing between the various vanity tokens.

Recommendations: Consider increasing the bytes allocated for each token in the extraData. Another option would be to instead store the leading 6 bytes of the hash of the token address. This would prevent unintentional clashes, since vanity addresses will still have very different hashes. The second approach however will not reduce the random chance of a 6 byte hash collision.

Customer's response: Fixed in be5f892.

Informational Issues

I-01. Forgotten Foundry import is still present

Description:

Inside `UniswapV4SettleAllStructVerifier.sol` there is a leftover import of Foundry's logging functions, probably imported during testing:

```JavaScript
import {CalldataDecoder} from "@univ4-periphery/src/libraries/CalldataDecoder.sol";
import {ICustomCondition} from "./interfaces/ICustomCondition.sol";
import {console2} from "@forge-std/console2.sol"; //@audit-issue unneeded import
import "./Lib.sol";
```

Recommendations: Remove the unnecessary import

Customer's response: Fixed in 1399d02.

I-02. Missing support for exact out swaps

Description: The verifier system features only an exact in swap verifier, seemingly intentionally not having an exact out path of usage, which could be inconvenient

Recommendations: Use the calldata decoder's method for decoding exact out swaps and introduce a verifier for this swap path as well

Customer's response: Fixed in 001492e.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.